Hệ thống nhúng

Tuần 1 — Tổng quan về Hệ thống nhúng

TS. Nguyễn Mạnh Hùng

Chương trình 9 tuần

Học phần: Hệ thống nhúng (Embedded Systems)

Mục tiêu buổi học

Chuẩn đầu ra liên quan

CLO1: Trình bày được khái niệm, đặc điểm, thị trường ứng dụng và quy trình thiết kế hệ thống nhúng.

Sau tuần 1, sinh viên có khả năng:

- Định nghĩa và phân biệt hệ thống nhúng với hệ thống máy tính đa năng.
- Nêu được các đặc điểm, phân loại và thị trường ứng dụng chính.
- Giải thích khái niệm Cyber-Physical Systems và vòng lặp điều khiển.
- So sánh các mô hình quy trình thiết kế: Waterfall, V-model, Spiral.
- Mô tả kiến trúc phần cứng tổng quát của một hệ thống nhúng.

Nội dung buổi học

Khái niệm và đặc điểm hệ thống nhúng

Cyber-Physical Systems và hệ thống điều khiển

Quy trình và phương pháp thiết kế

Kiến trúc phần cứng hệ thống nhúng

Tổng kết chương

Thực hành — Tuần 1

Khái niệm và đặc điểm hệ thống nhúng

Định nghĩa hệ thống nhúng

Hệ thống nhúng (Embedded System)

Là một hệ thống xử lý thông tin dựa trên máy tính (vi xử lý/vi điều khiển), được thiết kế để thực hiện một hoặc một số chức năng chuyên biệt, thường hoạt động như một thành phần bên trong một hệ thống lớn hơn.

- Khác với máy tính đa năng (PC), người dùng thường **không nhận biết** sự hiện diện của bộ xử lý bên trong.
- Ví dụ: bộ điều khiển động cơ ô tô, lò vi sóng, máy giặt, router Wi-Fi, máy đo huyết áp.

Hệ thống nhúng quanh ta

Ô tô
(ECU, ABS)

Gia dụng
(Máy giặt, lò VS)

Y tế
(Máy đo SpO2)

Viễn thông
(Router, Modem)

Công nghiệp
(PLC, Robot)

Tiêu dùng
(Smartwatch, TV)

Ước tính hơn 90% bộ vi xử lý sản xuất hàng năm trên thế giới được dùng trong hệ thống nhúng, không phải PC.

Hệ thống nhúng và máy tính đa năng

Tiêu chí	Hệ thống nhúng	Máy tính đa năng
Chức năng	Cố định, chuyên biệt	Đa nhiệm, linh hoạt
Tài nguyên	Giới hạn (bộ nhớ, năng lượng)	Dồi dào
Thời gian thực	Thường có yêu cầu chặt	Hiếm khi bắt buộc
Giao diện người dùng	Tối thiểu/không có	Đầy đủ (màn hình, bàn phím)
Vòng đời	Dài (nhiều năm)	Ngắn hơn, nâng cấp thường xuyên

Đặc điểm (1): Chuyên biệt hoá chức năng

- Mỗi hệ thống nhúng được thiết kế cho **một tập chức năng cố định**, đã biết trước tại thời điểm thiết kế.
- Cho phép tối ưu hoá sâu: giảm chi phí, giảm kích thước, tăng hiệu năng cho đúng tác vụ cần thiết.
- Đánh đổi: mất tính linh hoạt, khó/không thể thay đổi chức năng sau khi triển khai (trừ khi hỗ trợ cập nhật firmware).

Đặc điểm (2): Ràng buộc tài nguyên

- **Bộ nhớ**: thường chỉ vài KB–vài MB (so với GB của PC).
- **Năng lượng**: nhiều hệ thống chạy pin, cần tối ưu tiêu thụ điện.
- **Chi phí**: sản xuất số lượng lớn nên chi phí từng linh kiện phải tối thiểu.
- **Kích thước vật lý**: bị giới hạn bởi sản phẩm chứa nó.

Hệ quả thiết kế

Lập trình viên nhúng phải viết mã hiệu quả về bộ nhớ và năng lượng hơn nhiều so với lập trình ứng dụng thông thường.

Đặc điểm (3): Ràng buộc thời gian thực

- Nhiều hệ nhúng phải phản hồi trong **giới hạn thời gian xác định** (deadline).
- Không chỉ cần *đúng kết quả*, mà còn cần *đúng thời điểm*.
- Ví dụ: túi khí ô tô phải kích hoạt trong vài mili-giây sau va chạm.

Lưu ý

Khái niệm ”thời gian thực” **không đồng nghĩa** với ”nhanh” — mà là ”có thể dự đoán được” (predictable/deterministic).

Đặc điểm (4): Độ tin cậy và an toàn

- Nhiều hệ nhúng hoạt động liên tục, không giám sát, trong thời gian dài (năm).
- Lỗi phần mềm/phần cứng có thể gây hậu quả nghiêm trọng (an toàn con người, thiệt hại tài sản).
- Yêu cầu: khả năng chịu lỗi (fault tolerance), phục hồi (robustness), kiểm thử nghiêm ngặt.

Đặc điểm (5): Vòng đời sản phẩm

- Vòng đời có thể kéo dài **10–20 năm** (thiết bị công nghiệp, hàng không, y tế).
- Khó/tốn kém khi cập nhật hoặc thay thế sau khi triển khai.
- Cần thiết kế dự phòng: khả năng mở rộng, bảo trì, cập nhật firmware từ xa (OTA).

Phân loại theo tính thời gian thực

Loại	Mô tả	Ví dụ
Hard real-time	Trễ deadline = hệ thống thất bại hoàn toàn	Túi khí, hệ thống lái máy bay
Firm real-time	Trễ thỉnh thoảng chấp nhận được nhưng kết quả trễ vô giá trị	Xử lý video streaming
Soft real-time	Trễ làm giảm chất lượng nhưng vẫn còn giá trị	Giao diện người dùng

Ví dụ minh hoạ: Hard vs. Soft real-time

Hard real-time

Hệ thống phanh ABS: nếu tính toán chậm trễ dù chỉ vài ms, hệ thống có thể gây nguy hiểm trực tiếp.

Soft real-time

Ứng dụng phát nhạc Bluetooth: nếu trễ nhẹ, người nghe chỉ cảm thấy giật nhẹ, không nguy hiểm.

Phân loại theo quy mô và độ phức tạp

- **Hệ nhúng nhỏ (small-scale)**: 1 vi điều khiển 8/16-bit, phần mềm đơn giản (vài KB code). Vd: điều khiển từ xa.
- **Hệ nhúng trung bình (medium-scale)**: 1–2 bộ xử lý 16/32-bit, có thể có RTOS. Vd: máy in.
- **Hệ nhúng phức tạp / kết nối mạng (complex/networked)**: nhiều bộ xử lý, giao tiếp mạng, xử lý song song. Vd: ô tô hiện đại (hàng chục ECU).

Thị trường ứng dụng: Ô tô

- Xe hơi hiện đại chứa **50–100+ ECU** (Electronic Control Unit).
- Ứng dụng: động cơ, phanh ABS, túi khí, hệ thống giải trí, hỗ trợ lái (ADAS).
- Giao tiếp giữa các ECU qua mạng CAN bus, LIN, FlexRay, Automotive Ethernet.
- Xu hướng: xe điện (EV), xe tự hành (autonomous driving).

Thị trường ứng dụng: Y tế

- Thiết bị đeo: máy đo nhịp tim, SpO2, đồng hồ thông minh theo dõi sức khoẻ.
- Thiết bị cấy ghép: máy tạo nhịp tim (pacemaker), bơm insulin tự động.
- Thiết bị chẩn đoán: máy siêu âm cầm tay, máy xét nghiệm tại chỗ (point-of-care).
- Yêu cầu đặc biệt cao về **an toàn** và **độ tin cậy** (chuẩn IEC 62304).

Thị trường ứng dụng: Công nghiệp và Robot

- PLC (Programmable Logic Controller) điều khiển dây chuyền sản xuất.
- Robot công nghiệp: điều khiển động cơ chính xác cao, cảm biến vị trí.
- Hệ thống SCADA giám sát và thu thập dữ liệu quy mô lớn.
- Xu hướng Industry 4.0: nhà máy thông minh, bảo trì dự đoán (predictive maintenance).

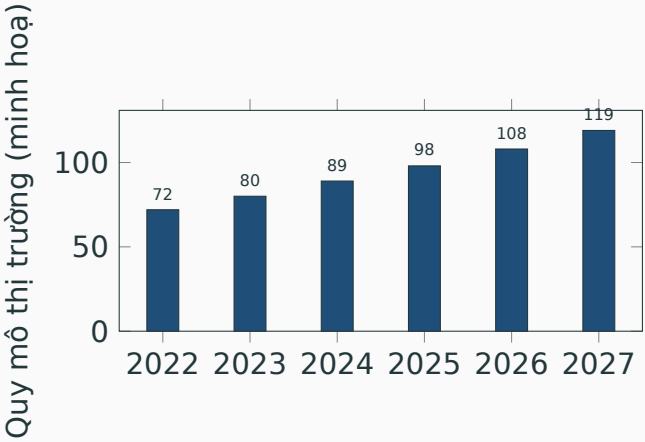
Thị trường ứng dụng: Điện tử tiêu dùng và IoT

- Điện thoại thông minh, TV, loa thông minh, thiết bị nhà thông minh.
- IoT (Internet of Things): hàng tỷ thiết bị kết nối, thu thập và trao đổi dữ liệu.
- Đặc điểm: chi phí cực thấp, tiêu thụ năng lượng cực thấp, kết nối không dây (Wi-Fi, BLE, Zigbee, LoRa).

Thị trường ứng dụng: Hàng không và Quốc phòng

- Hệ thống điều khiển bay (fly-by-wire), radar, dẫn đường (navigation).
- Yêu cầu chứng nhận nghiêm ngặt (DO-178C cho phần mềm hàng không).
- Độ tin cậy cực cao: dự phòng đa lớp (redundancy), khả năng chịu lỗi.

Xu hướng thị trường hệ thống nhúng



Số liệu minh hoạ xu hướng tăng trưởng; sinh viên nên tra cứu báo cáo thị trường cập nhật (Statista, IDC, Gartner) khi cần số liệu chính xác.

Cơ hội nghề nghiệp ngành hệ thống nhúng

- **Kỹ sư phần cứng nhúng**: thiết kế mạch, chọn linh kiện, PCB.
- **Kỹ sư phần mềm nhúng (firmware)**: lập trình C/C++/Assembly, driver, RTOS.
- **Kỹ sư kiểm thử nhúng**: kiểm thử HIL (Hardware-in-the-loop), automation.
- **Kỹ sư hệ thống**: tích hợp phần cứng–phần mềm, quản lý yêu cầu.
- Kỹ năng nền tảng: C/C++, vi xử lý, điện tử số, hệ điều hành thời gian thực.

Kiểm tra nhanh — Câu hỏi

Thảo luận

1. Một máy giặt và một máy tính xách tay: đâu là hệ thống nhúng? Vì sao?
2. Nêu 2 ví dụ hệ thống *hard real-time* khác ngoài túi khí ô tô.

Kiểm tra nhanh — Gợi ý đáp án

1. Máy giặt là hệ thống nhúng: vi điều khiển thực hiện chức năng cố định (điều khiển chu trình giặt), người dùng không thấy ”máy tính” bên trong. Laptop là máy tính đa năng.
2. Ví dụ hard real-time: hệ thống điều khiển bay (fly-by-wire), máy tạo nhịp tim, hệ thống phanh khẩn cấp tàu hoả.

Tổng kết: Khái niệm và đặc điểm

- Hệ thống nhúng = máy tính chuyên dụng, ẩn bên trong sản phẩm lớn hơn.
- 5 đặc điểm chính: chuyên biệt hoá, ràng buộc tài nguyên, thời gian thực, độ tin cậy, vòng đời dài.
- Phân loại theo thời gian thực (hard/firm/soft) và theo quy mô.
- Ứng dụng trải rộng: ô tô, y tế, công nghiệp, tiêu dùng, hàng không.

Cyber-Physical Systems và hệ thống điều khiển

Định nghĩa Cyber-Physical Systems (CPS)

CPS

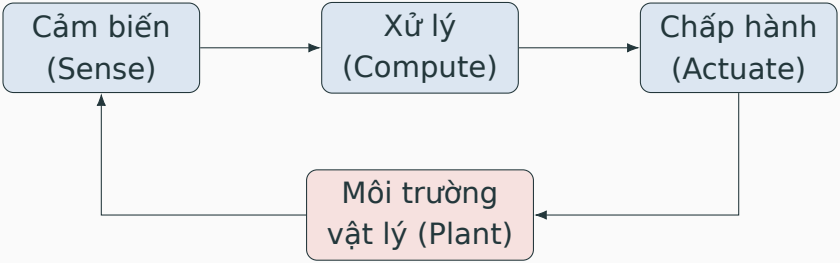
Hệ thống tích hợp chặt chẽ giữa **tính toán (cyber)** và **quá trình vật lý (physical)**, trong đó phần tính toán giám sát và điều khiển các quá trình vật lý, thường qua vòng phản hồi (feedback loop).

- CPS là bước phát triển tiếp theo của hệ thống nhúng truyền thống, nhấn mạnh sự **tương tác với môi trường vật lý** qua cảm biến/cơ cấu chấp hành.

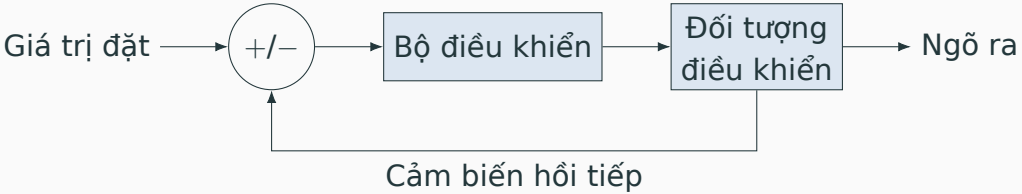
Ba thành phần cốt lõi của CPS

- **Compute** — Tính toán: vi điều khiển/vi xử lý thực thi thuật toán điều khiển.
- **Communication** — Truyền thông: trao đổi dữ liệu giữa các thành phần, có thể qua mạng.
- **Physical** — Vật lý: cảm biến (sensor) thu thập, cơ cấu chấp hành (actuator) tác động lên thế giới thực.

Vòng lặp Sense – Compute – Actuate



Vòng điều khiển hồi tiếp (Feedback Control Loop)



Open-loop và Closed-loop control

	Open-loop	**Closed-loop**
Hồi tiếp	Không có	Có
Độ chính xác	Phụ thuộc mô hình	Tự điều chỉnh sai số
Độ phức tạp	Thấp	Cao hơn
Ví dụ	Lò vi sóng hẹn giờ cố định	Điều hoà nhiệt độ

Ví dụ Open-loop: Lò vi sóng hẹn giờ

- Người dùng đặt thời gian nấu 3 phút.
- Hệ thống phát nhiệt trong đúng 3 phút, **không kiểm tra** nhiệt độ thực tế của thức ăn.
- Đơn giản, rẻ, nhưng không tự điều chỉnh nếu điều kiện thay đổi.

Ví dụ Closed-loop: Điều hoà nhiệt độ

- Cảm biến liên tục đo nhiệt độ phòng.
- Bộ điều khiển so sánh với nhiệt độ đặt, quyết định bật/tắt máy nén.
- Hệ thống **tự điều chỉnh** theo sai số thực tế → chính xác hơn open-loop.

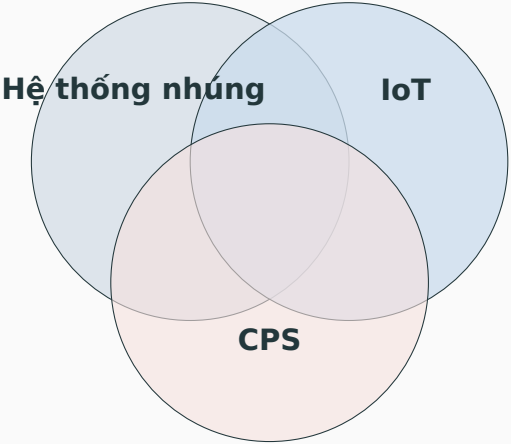
CPS trong Công nghiệp 4.0

- Nhà máy thông minh: máy móc kết nối, tự giám sát trạng thái (condition monitoring).
- Bảo trì dự đoán: phân tích dữ liệu cảm biến để dự đoán hỏng hóc trước khi xảy ra.
- Digital twin: mô hình số song song với hệ thống vật lý thực.

CPS trong giao thông thông minh

- Xe tự hành: tích hợp cảm biến (radar, lidar, camera), xử lý AI thời gian thực, điều khiển động cơ/phanh/lái.
- Hệ thống giao thông thông minh (ITS): đèn tín hiệu thích ứng theo lưu lượng.
- V2X (Vehicle-to-Everything): xe giao tiếp với xe khác, hạ tầng, người đi bộ.

Quan hệ CPS – IoT – Hệ thống nhúng



Hệ thống nhúng là nền tảng công nghệ; IoT nhấn mạnh kết nối mạng; CPS nhấn mạnh vòng lặp tương tác chặt với thế giới vật lý. Ba khái niệm giao thoa và bổ sung lẫn nhau.

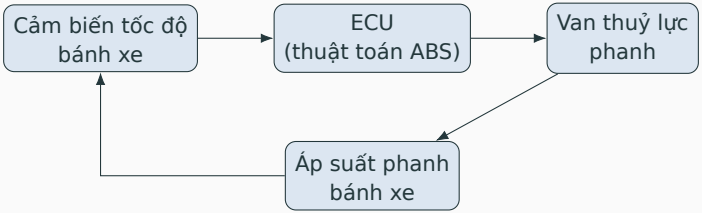
Thách thức thiết kế CPS: An toàn (Safety)

- Lỗi hệ thống có thể gây tổn hại con người/môi trường.
- Cần phân tích rủi ro (hazard analysis), thiết kế dự phòng (redundancy), tuân thủ tiêu chuẩn an toàn (ISO 26262 cho ô tô, IEC 61508 tổng quát).

Thách thức thiết kế CPS: Bảo mật (Security)

- CPS kết nối mạng → bề mặt tấn công (attack surface) lớn hơn hệ nhúng độc lập.
- Rủi ro: chiếm quyền điều khiển từ xa, đánh cắp dữ liệu, tấn công từ chối dịch vụ.
- Biện pháp: mã hoá, xác thực, Secure Boot, cập nhật firmware an toàn.

Case study: Hệ thống phanh ABS



ECU liên tục nhận tín hiệu tốc độ bánh xe; nếu phát hiện bánh sắp bó cứng (nguy cơ trượt), điều chỉnh nhanh áp suất thuỷ lực — vòng lặp lặp lại hàng chục lần/giây.

Kiểm tra nhanh — Câu hỏi

Thảo luận

1. Máy pha cà phê tự động có phải là CPS không? Vì sao?
2. Nêu một rủi ro bảo mật cụ thể nếu một khoá thông minh (smart lock) bị tấn công.

Kiểm tra nhanh — Gợi ý đáp án

1. Có, nếu nó có cảm biến (mực nước, nhiệt độ) và vòng phản hồi điều chỉnh quá trình pha; nếu chỉ hẹn giờ cố định thì thiên về open-loop đơn giản hơn là CPS đầy đủ.
2. Kẻ tấn công có thể mở khoá từ xa trái phép, xâm nhập nhà — minh hoạ tầm quan trọng của bảo mật trong CPS kết nối mạng.

Tổng kết: CPS và hệ thống điều khiển

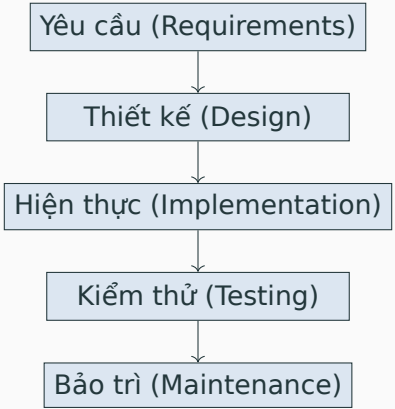
- CPS = tích hợp chặt giữa tính toán và thế giới vật lý qua vòng lặp Sense-Compute-Actuate.
- Closed-loop control cho phép hệ thống tự điều chỉnh theo sai số thực tế.
- CPS là nền tảng của Công nghiệp 4.0, xe tự hành, giao thông thông minh.
- Hai thách thức lớn: an toàn (safety) và bảo mật (security).

Quy trình và phương pháp thiết kế

Vòng đời phát triển sản phẩm nhúng

- Thu thập yêu cầu (requirements) $\rightarrow$ Thiết kế (design) $\rightarrow$ Hiện thực hoá (implementation) $\rightarrow$ Kiểm thử (testing) $\rightarrow$ Triển khai (deployment) $\rightarrow$ Bảo trì (maintenance).
- Khác với phần mềm thuần tuý: nhúng cần đồng thời quản lý cả **phần cứng** và **phần mềm**, chi phí sửa lỗi sau sản xuất rất cao.

Mô hình Waterfall



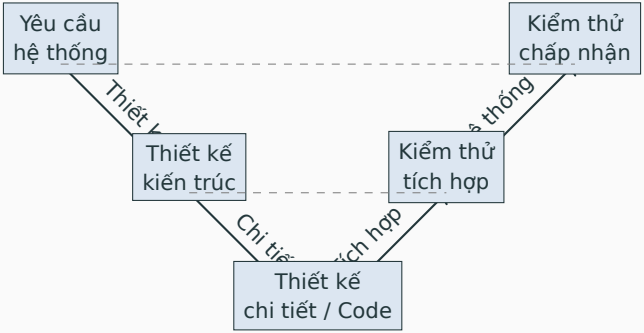
Waterfall — Ưu điểm

- Cấu trúc rõ ràng, dễ quản lý theo từng giai đoạn.
- Tài liệu hoá đầy đủ ở mỗi bước — phù hợp yêu cầu pháp lý/chứng nhận.
- Dễ ước lượng thời gian, chi phí ngay từ đầu nếu yêu cầu ổn định.

Waterfall — Nhược điểm

- Khó quay lại sửa đổi khi phát hiện lỗi ở giai đoạn sau.
- Không phù hợp khi yêu cầu thay đổi liên tục.
- Khách hàng chỉ thấy sản phẩm ở giai đoạn rất muộn.

Mô hình V-model



V-model — Ánh xạ thiết kế và kiểm thử

- Mỗi giai đoạn **thiết kế** bên trái có một giai đoạn **kiểm thử** tương ứng bên phải.
- Yêu cầu hệ thống $\leftrightarrow$ Kiểm thử chấp nhận (Acceptance test).
- Thiết kế kiến trúc $\leftrightarrow$ Kiểm thử tích hợp (Integration test).
- Thiết kế chi tiết $\leftrightarrow$ Kiểm thử đơn vị (Unit test).
- Ưu điểm: kế hoạch kiểm thử được chuẩn bị song song từ sớm, giảm rủi ro phát hiện lỗi muộn.

Mô hình Spiral



Mỗi vòng xoáy = 1 chu kỳ lặp: xác định mục tiêu $\rightarrow$ đánh giá rủi ro $\rightarrow$ phát triển $\rightarrow$ lập kế hoạch vòng tiếp theo.

Spiral — Khi nào phù hợp?

- Dự án có **rủi ro kỹ thuật cao**, yêu cầu chưa rõ ràng hoàn toàn.
- Cho phép đánh giá và giảm thiểu rủi ro sau mỗi vòng lặp.
- Chi phí quản lý cao hơn Waterfall — phù hợp dự án lớn, phức tạp.

Agile/Iterative trong phát triển nhúng

- Phát triển theo các chu kỳ ngắn (sprint), tích hợp và kiểm thử liên tục.
- Thách thức khi áp dụng cho nhúng: phần cứng không ”linh hoạt” như phần mềm thuần tuý (thời gian sản xuất PCB, mua linh kiện).
- Giải pháp thực tế: kết hợp Agile cho phần mềm + kế hoạch giai đoạn cho phần cứng (hybrid).

So sánh các mô hình quy trình

Mô hình	Ưu điểm chính	Nhược điểm chính	Phù hợp
Waterfall	Rõ ràng, dễ quản lý	Cứng nhắc	Yêu cầu ổn định
V-model	Kiểm thử song song thiết kế	Vẫn tuần tự	Hệ thống an toàn cao
Spiral	Giảm rủi ro theo vòng lặp	Chi phí quản lý cao	Rủi ro kỹ thuật cao
Agile	Linh hoạt, phản hồi nhanh	Khó áp dụng thuần cho HW	Phần mềm thay đổi nhiều

Tiêu chí lựa chọn mô hình phát triển

- Mức độ rõ ràng của yêu cầu ngay từ đầu.
- Mức độ an toàn/chứng nhận bắt buộc (hàng không, y tế → thường dùng V-model).
- Quy mô đội ngũ và kinh nghiệm.
- Ràng buộc thời gian, ngân sách.

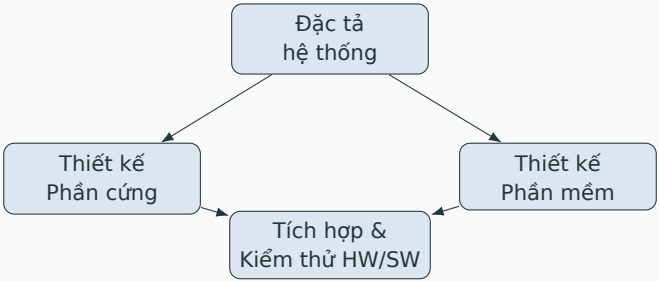
Đồng thiết kế phần cứng – phần mềm (HW/SW Co-design)

Khái niệm

Quá trình thiết kế đồng thời cả phần cứng và phần mềm của hệ thống nhúng, thay vì thiết kế phần cứng xong rồi mới viết phần mềm.

- Mục tiêu: tối ưu hoá tổng thể hệ thống (hiệu năng, chi phí, năng lượng).
- Cho phép đánh giá đánh đổi (trade-off) HW/SW sớm trong quá trình thiết kế.

Quy trình HW/SW Co-design



Bài toán phân chia HW/SW (Partitioning)

- Quyết định chức năng nào thực hiện bằng **phần cứng chuyên dụng** (nhanh, tốn diện tích/chi phí) hay **phần mềm** (linh hoạt, chậm hơn).
- Tiêu chí: yêu cầu hiệu năng, tính linh hoạt cần thiết, chi phí, thời gian ra thị trường.
- Ví dụ: xử lý tín hiệu tốc độ cao → FPGA/ASIC; logic điều khiển phức tạp → vi điều khiển + phần mềm.

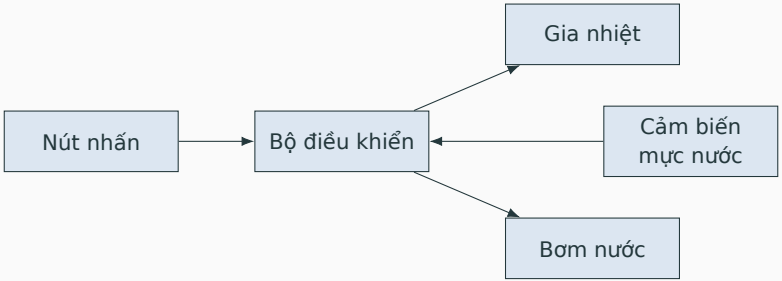
Đặc tả yêu cầu hệ thống

- Bước đầu tiên và quan trọng nhất: xác định **hệ thống cần làm gì**.
- Yêu cầu cần: rõ ràng, đo lường được, không mâu thuẫn, có thể kiểm chứng (verifiable).
- Công cụ: tài liệu đặc tả yêu cầu (SRS), use-case, user story.

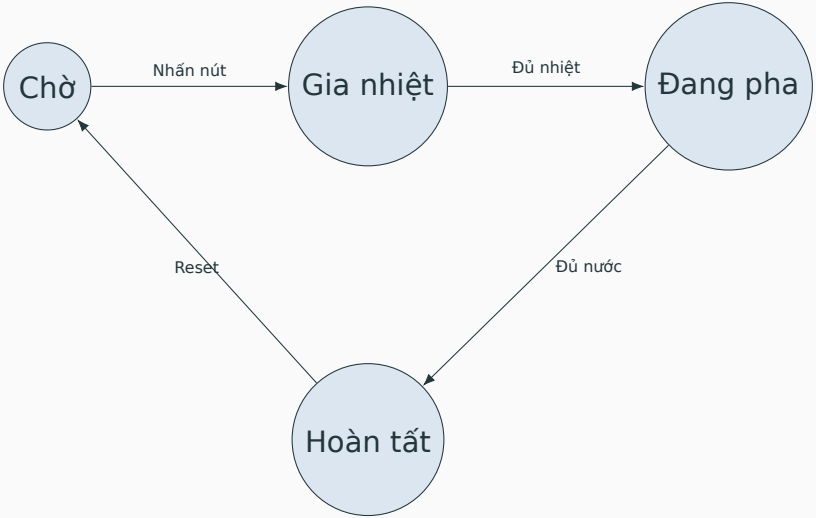
Yêu cầu chức năng và phi chức năng

Yêu cầu chức năng	Yêu cầu phi chức năng
Hệ thống PHẢI làm gì Vd: ”Đọc giá trị cảm biến mỗi 100ms”	Hệ thống hoạt động TỐT như thế nào Vd: ”Độ trễ phản hồi $<$ 10ms”, ”Tiêu thụ $<$ 50mA”

Sơ đồ khối chức năng — Ví dụ: Máy pha cà phê



Sơ đồ trạng thái hữu hạn (FSM) — Máy pha cà phê



Kỹ thuật đặc tả khác: Lưu đồ (Flowchart)

- Biểu diễn trình tự các bước xử lý bằng ký hiệu hình học chuẩn (bắt đầu/kết thúc, xử lý, quyết định, I/O).
- Trực quan, dễ hiểu cho cả người không chuyên kỹ thuật.
- Hạn chế: khó biểu diễn hành vi song song, phức tạp khi hệ thống lớn.

Kỹ thuật đặc tả khác: UML/SysML

- UML (Unified Modeling Language): biểu đồ lớp, biểu đồ tuần tự, biểu đồ trạng thái — phổ biến cho phần mềm.
- SysML: mở rộng UML cho kỹ thuật hệ thống (bao gồm cả phần cứng, yêu cầu).
- Hỗ trợ thiết kế theo mô hình (Model-Based Design), sinh code tự động từ mô hình.

Prototype và kiểm thử lặp

- Xây dựng bản mẫu (prototype) sớm để xác thực ý tưởng thiết kế.
- Kiểm thử lặp lại giúp phát hiện lỗi sớm, giảm chi phí sửa chữa.
- Công cụ hỗ trợ: breadboard, kit phát triển (development board), mô phỏng phần mềm.

Quản lý dự án nhúng: các mốc quan trọng

- **Concept freeze**: chốt ý tưởng/yêu cầu chính.
- **Design freeze**: chốt thiết kế phần cứng (trước khi đặt PCB).
- **Code freeze**: ngừng thêm tính năng mới, chỉ sửa lỗi.
- **Release**: phát hành sản phẩm chính thức.

Kiểm tra nhanh — Câu hỏi

Thảo luận

1. Vì sao hệ thống điều khiển máy bay thường chọn V-model thay vì Agile thuần tuý?
2. HW/SW co-design giúp ích gì so với thiết kế phần cứng xong mới viết phần mềm?

Kiểm tra nhanh — Gợi ý đáp án

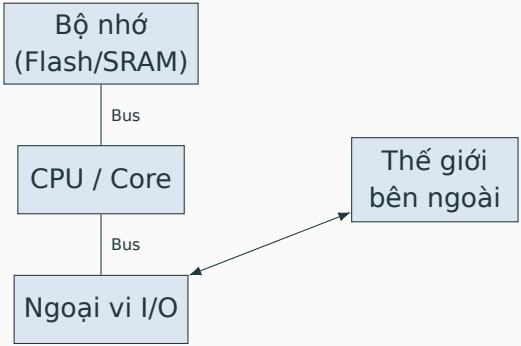
1. Hàng không yêu cầu chứng nhận nghiêm ngặt (DO-178C), cần tài liệu hoá và kiểm thử đầy đủ tại mỗi giai đoạn — V-model đáp ứng tốt hơn Agile thuần tuý vốn ưu tiên thay đổi nhanh, ít tài liệu.
2. Co-design phát hiện sớm các đánh đổi (vd: một chức năng nên làm bằng phần cứng hay phần mềm), tránh phải thiết kế lại phần cứng tốn kém khi phát hiện muộn.

Tổng kết: Quy trình và phương pháp thiết kế

- Ba mô hình quy trình kinh điển: Waterfall, V-model, Spiral — mỗi mô hình phù hợp bối cảnh khác nhau.
- HW/SW co-design giúp tối ưu hoá đồng thời phần cứng và phần mềm.
- Đặc tả yêu cầu (functional/non-functional) là nền tảng của mọi quy trình thiết kế.
- Công cụ đặc tả: sơ đồ khối, FSM, flowchart, UML/SysML.

Kiến trúc phần cứng hệ thống nhúng

Sơ đồ khối tổng quát hệ thống nhúng



Đơn vị xử lý trung tâm (CPU/Core)

- Thành phần: ALU (tính toán số học/logic), thanh ghi (registers), bộ điều khiển (control unit).
- Thực thi chu trình **Fetch – Decode – Execute** lặp lại liên tục.
- Tốc độ đo bằng tần số xung nhịp (clock frequency, MHz).

Phân loại bộ xử lý: Microprocessor

- Chỉ chứa CPU — cần chip nhớ và ngoại vi **rời** bên ngoài.
- Hiệu năng cao, linh hoạt về cấu hình hệ thống.
- Ví dụ: Intel x86, ARM Cortex-A (dùng trong PC, điện thoại, hệ thống nhúng cao cấp chạy Linux/Android).

Phân loại bộ xử lý: Microcontroller (MCU)

- Tích hợp CPU + bộ nhớ (Flash/SRAM) + ngoại vi (GPIO, Timer, ADC, UART...) trên **một chip duy nhất**.
- Chi phí thấp, tiêu thụ năng lượng thấp, phù hợp ứng dụng nhúng phổ thông.
- Ví dụ: họ AVR (ATmega), PIC, STM32 (ARM Cortex-M) — đây là đối tượng chính của học phần này.

Phân loại bộ xử lý: DSP

- DSP (Digital Signal Processor): tối ưu cho xử lý tín hiệu số tốc độ cao (âm thanh, hình ảnh, viễn thông).
- Có tập lệnh chuyên dụng cho phép toán nhân-cộng-dồn (MAC) tốc độ cao.
- Ví dụ: xử lý âm thanh trong tai nghe khử tiếng ồn, modem.

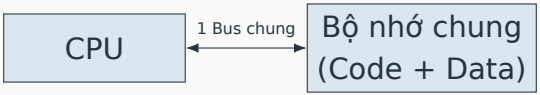
Phân loại bộ xử lý: FPGA và SoC

- **FPGA** (Field-Programmable Gate Array): mạch logic có thể cấu hình lại, xử lý song song thực sự ở mức phần cứng.
- **SoC** (System-on-Chip): tích hợp CPU + FPGA/DSP + ngoại vi phức tạp trên một chip (vd: Xilinx Zynq).
- Phù hợp ứng dụng cần hiệu năng cực cao hoặc xử lý song song đặc biệt.

So sánh Microprocessor và Microcontroller

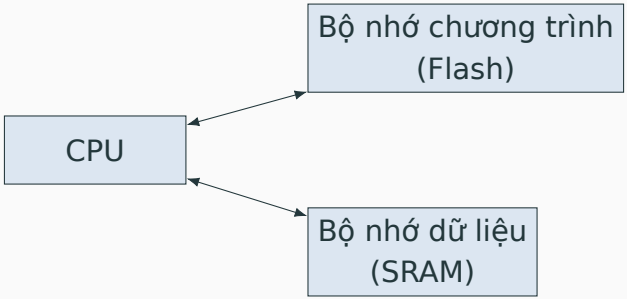
Tiêu chí	Microprocessor	Microcontroller
Tích hợp	Chỉ CPU	CPU + Bộ nhớ + Ngoại vi
Chi phí hệ thống	Cao hơn (cần linh kiện ngoài)	Thấp hơn
Hiệu năng	Cao	Vừa phải, đủ dùng
Ứng dụng điển hình	PC, máy chủ, điện thoại	Thiết bị nhúng phổ thông

Kiến trúc Von Neumann



Chương trình và dữ liệu chia sẻ **cùng một không gian bộ nhớ** và cùng một bus truy cập.

Kiến trúc Harvard



Bộ nhớ chương trình và dữ liệu **tách biệt**, mỗi loại có bus riêng → truy cập đồng thời, nhanh hơn. Vi điều khiển AVR sử dụng kiến trúc này.

So sánh Von Neumann và Harvard

	Von Neumann	Harvard
Bộ nhớ	Chung code+data	Tách riêng
Bus	1 bus chung	2 bus độc lập
Tốc độ	Bị nghẽn cổ chai (bottle-neck)	Truy cập song song, nhanh hơn
Ví dụ	x86 truyền thống	AVR, PIC, ARM (bộ nhớ cache)

Phân cấp bộ nhớ trong hệ nhúng

- **Flash**: lưu chương trình (firmware), không mất dữ liệu khi mất điện (non-volatile), tốc độ ghi chậm hơn đọc.
- **SRAM**: lưu dữ liệu tạm thời (biến, ngăn xếp), tốc độ nhanh, mất dữ liệu khi mất điện (volatile).
- **EEPROM**: lưu dữ liệu cấu hình cần giữ lại lâu dài, ghi/xoá được nhiều lần nhưng chậm và có giới hạn chu kỳ ghi.

Flash Memory

- Lưu trữ mã chương trình (program code) của vi điều khiển.
- Non-volatile: giữ dữ liệu ngay cả khi mất nguồn.
- Giới hạn số chu kỳ ghi/xoá (thường 10.000–100.000 lần) — không nên ghi Flash liên tục trong vòng lặp chính.

SRAM

- Static RAM: lưu biến toàn cục, biến cục bộ, ngăn xếp (stack), heap.
- Tốc độ truy cập rất nhanh, không giới hạn chu kỳ ghi.
- Volatile: mất toàn bộ dữ liệu khi mất nguồn — cần cơ chế lưu trạng thái nếu cần giữ dữ liệu qua các lần khởi động lại.

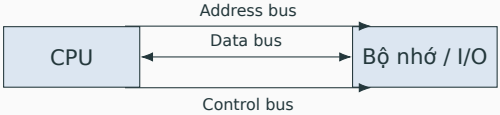
EEPROM

- Electrically Erasable Programmable ROM: ghi/xoá được bằng phần mềm, từng byte.
- Non-volatile, thường dùng lưu tham số cấu hình (vd: địa chỉ thiết bị, số liệu hiệu chuẩn).
- Tốc độ ghi chậm hơn nhiều so với SRAM; giới hạn chu kỳ ghi (100.000 lần/ô nhớ).

So sánh Flash / SRAM / EEPROM

	Flash	SRAM	EEPROM
Vai trò	Mã chương trình	Dữ liệu tạm	Cấu hình lâu dài
Non-volatile?	Có	Không	Có
Tốc độ	Đọc nhanh, ghi chậm	Rất nhanh	Chậm
Chu kỳ ghi	Giới hạn	Không giới hạn	Giới hạn

Bus hệ thống



- **Address bus**: CPU chỉ định địa chỉ ô nhớ/thanh ghi cần truy cập (một chiều, từ CPU).
- **Data bus**: truyền dữ liệu thực tế (hai chiều).
- **Control bus**: tín hiệu điều khiển (đọc/ghi, clock, ngắt…).

Tổng quan ngoại vi: GPIO

- GPIO (General-Purpose Input/Output): chân số có thể cấu hình là ngõ vào hoặc ngõ ra.
- Ngõ ra: điều khiển LED, relay, transistor điều khiển tải.
- Ngõ vào: đọc trạng thái nút nhấn, công tắc, cảm biến số.

Tổng quan ngoại vi: Timer/Counter

- Bộ đếm phần cứng hoạt động độc lập với chương trình chính (CPU).
- Ứng dụng: đo thời gian, tạo trễ chính xác, tạo xung PWM, đếm sự kiện bên ngoài.
- Sẽ tìm hiểu chi tiết ở Tuần 5.

Tổng quan ngoại vi: ADC/DAC

- **ADC** (Analog-to-Digital Converter): chuyển tín hiệu tương tự (điện áp cảm biến) sang giá trị số.
- **DAC** (Digital-to-Analog Converter): chuyển giá trị số sang tín hiệu tương tự.
- Sẽ tìm hiểu chi tiết ở Tuần 6.

Tổng quan ngoại vi: UART/SPI/I2C

- Các chuẩn giao tiếp nối tiếp phổ biến để kết nối vi điều khiển với cảm biến, module ngoại vi khác.
- UART: giao tiếp 2 dây, không đồng bộ. SPI: tốc độ cao, đồng bộ. I2C: đa thiết bị trên 2 dây.
- Sẽ tìm hiểu chi tiết ở Tuần 6.

Giới thiệu họ vi điều khiển AVR

- Họ vi điều khiển 8-bit của Atmel (nay thuộc Microchip), kiến trúc Harvard, RISC.
- Dòng phổ biến: **ATmega328P** (dùng trong Arduino Uno), ATmega169 (AVR Butterfly), ATmega2560 (Arduino Mega).
- Đặc điểm: tập lệnh đơn giản, hầu hết lệnh thực thi trong 1 chu kỳ xung nhịp — hiệu năng cao so với xung nhịp thực tế.

Board thực hành: Arduino Uno

Thông số	Giá trị
Vi điều khiển	ATmega328P
Tần số xung nhịp	16 MHz
Bộ nhớ Flash	32 KB (0.5 KB bootloader)
SRAM	2 KB
EEPROM	1 KB
Chân I/O số	14 (6 hỗ trợ PWM)
Chân vào tương tự	6 (ADC 10-bit)
Điện áp hoạt động	5V

Sơ đồ chân Arduino Uno / ATmega328P (tóm tắt)

Nhóm chân	Chức năng
D0–D13	GPIO số; D0/D1 = UART (RX/TX); D3,5,6,9,10,11 = PWM
A0–A5	Ngõ vào tương tự (ADC), có thể dùng như GPIO số
5V, 3.3V, GND	Chân nguồn
RESET	Khởi động lại vi điều khiển

Nguồn cấp và tiêu thụ năng lượng

- Vi điều khiển thường hoạt động ở 3.3V hoặc 5V.
- Dòng tiêu thụ ở chế độ hoạt động: vài mA đến vài chục mA tuỳ tần số xung nhịp.
- Thiết kế nhúng chạy pin cần tối ưu hoá năng lượng ở cả phần cứng lẫn phần mềm.

Chế độ tiết kiệm năng lượng (Sleep modes)

- Vi điều khiển AVR hỗ trợ nhiều chế độ ngủ: Idle, ADC Noise Reduction, Power-down, Power-save, Standby.
- Power-down: tắt gần như toàn bộ, dòng tiêu thụ chỉ còn vài μA, đánh thức bằng ngắt ngoài.
- Sẽ vận dụng khi thiết kế ứng dụng chạy pin dài ngày.

Xu hướng phần cứng nhúng hiện đại

- Chuyển dịch từ 8-bit (AVR, PIC) sang 32-bit (ARM Cortex-M) cho ứng dụng cần hiệu năng cao hơn.
- Tích hợp kết nối không dây trực tiếp trên chip (Wi-Fi, BLE) — vd: ESP32.
- Xu hướng AI-on-edge: chạy mô hình học máy nhỏ ngay trên vi điều khiển (TinyML).

Kiểm tra nhanh — Câu hỏi

Thảo luận

1. Vì sao AVR dùng kiến trúc Harvard thay vì Von Neumann?
2. Nếu cần lưu lại số lần thiết bị được bật, nên dùng Flash, SRAM hay EEPROM? Vì sao?

Kiểm tra nhanh — Gợi ý đáp án

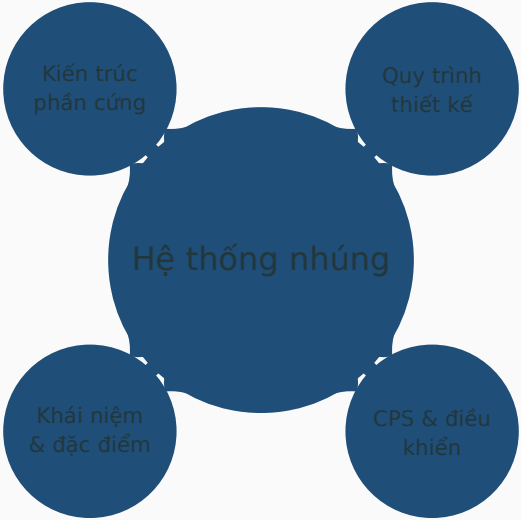
1. Kiến trúc Harvard cho phép CPU truy cập đồng thời bộ nhớ chương trình và dữ liệu qua 2 bus riêng biệt, tăng tốc độ thực thi — quan trọng với vi điều khiển tần số thấp cần hiệu năng tối đa trên mỗi chu kỳ xung nhịp.
2. Nên dùng EEPROM: cần non-volatile (giữ được qua các lần mất điện) và số lần ghi không quá thường xuyên (chỉ ghi khi tắt/mở thiết bị) — phù hợp giới hạn chu kỳ ghi của EEPROM.

Tổng kết: Kiến trúc phần cứng hệ thống nhúng

- Hệ thống nhúng = CPU + Bộ nhớ (Flash/SRAM/EEPROM) + Ngoại vi, kết nối qua bus.
- Vi điều khiển tích hợp mọi thành phần trên 1 chip — khác với vi xử lý cần linh kiện rời.
- Kiến trúc Harvard (bus tách biệt) phổ biến trong vi điều khiển như AVR.
- Arduino Uno / ATmega328P là nền tảng thực hành chính của học phần.

Tổng kết chương

Bản đồ tư duy — Tuần 1



Lộ trình các tuần tiếp theo

- **Tuần 2**: Kiến trúc và tập lệnh vi điều khiển AVR.
- **Tuần 3**: Lập trình C cho hệ thống nhúng.
- **Tuần 4**: Bộ nhớ, Bus dữ liệu và tối ưu hoá.
- **Tuần 5**: Ngắt, Timer/Counter và Watchdog.
- **Tuần 6–8**: Giao tiếp nối tiếp, RTOS, điều khiển & tổng kết.

Chuẩn đầu ra đã đạt được

CLO1 — Hoàn thành

Sinh viên đã có khả năng trình bày khái niệm, đặc điểm, thị trường ứng dụng và quy trình thiết kế hệ thống nhúng.

Nội dung này là nền tảng cho toàn bộ các tuần tiếp theo của học phần.

Tài liệu tham khảo — Tuần 1

- James K. Peckol, *Embedded Systems: A Contemporary Design Tool*, 2nd Ed., Wiley, 2019.
- Ryan Robucci, *CMPE 311 — Lecture 00: Introduction to Embedded Systems*, UMBC.
- Phillip Koopman, *18-348 — L01: Course Overview*, *L02: Embedded Hardware*, CMU.

Thực hành — Tuần 1

Mục tiêu buổi thực hành

- Cài đặt thành công môi trường phát triển cho vi điều khiển AVR.
- Làm quen giao diện lập trình, biên dịch và nạp chương trình.
- Thực hiện thành công bài Lab đầu tiên: chớp tắt LED.

Chuẩn bị: Cài đặt môi trường phát triển

1. Cài đặt Arduino IDE (`arduino.cc`) hoặc Microchip Studio (dành cho lập trình AVR gần phần cứng hơn).
2. Cài đặt driver USB cho board Arduino Uno (thường tự động trên Windows 10+/macOS/Linux).
3. Kiểm tra cài đặt: mở IDE, chọn đúng *Board* và *Port*.

Làm quen giao diện IDE

- Vùng soạn thảo mã nguồn (sketch/code editor).
- Nút **Verify** (biên dịch kiểm tra lỗi) và **Upload** (nạp chương trình vào board).
- Cửa sổ **Serial Monitor**: xem dữ liệu gửi từ board qua UART.

Kết nối board với máy tính

1. Cắm cáp USB từ Arduino Uno vào máy tính.
2. Trong IDE: `Tools > Board > Arduino Uno`.
3. Trong IDE: `Tools > Port` — chọn cổng COM/tty tương ứng.
4. Đèn nguồn (ON) trên board sáng — xác nhận board được cấp điện.

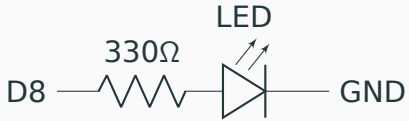
Bài Lab 1: Chớp tắt LED — Đề bài

Yêu cầu

Viết chương trình điều khiển LED nối vào chân D8 nhấp nháy với chu kỳ 1 giây sáng / 1 giây tắt.

- Sử dụng thanh ghi `DDRB`, `PORTB` (lập trình thanh ghi trực tiếp) hoặc hàm `pinMode()`, `digitalWrite()` (thư viện Arduino).
- Nộp báo cáo: mã nguồn + video/ảnh chụp kết quả.

Bài Lab 1: Sơ đồ đấu nối



Điện trở giới hạn dòng (≈ 330Ω) nối tiếp với LED để bảo vệ chân vi điều khiển.

Bài Lab 1: Code mẫu (thanh ghi trực tiếp)

```c
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    DDRB |= (1 << PB0);      // D8 = PB0: cau hinh la output
    while (1) {
        PORTB |= (1 << PB0);  // Bat LED
        _delay_ms(1000);
        PORTB &= ~(1 << PB0); // Tat LED
        _delay_ms(1000);
    }
}
```

Sử dụng bộ mô phỏng và debugger cơ bản

- Nếu chưa có board thật: dùng mô phỏng trực tuyến **Wokwi** hoặc phần mềm **Proteus/SimulIDE**.
- Debugger cơ bản: đặt điểm dừng (breakpoint), theo dõi giá trị thanh ghi/biến trong Microchip Studio.
- Kỹ năng debug sẽ được rèn luyện sâu hơn ở các tuần sau.

Bài tập về nhà (tự luyện)

1. Thay đổi chu kỳ nhấp nháy LED thành 0.5 giây sáng / 0.5 giây tắt.
2. Kết nối thêm 1 LED thứ hai (chân D9), lập trình 2 LED nhấp nháy so le (một sáng thì một tắt).
3. Viết sơ đồ khối mô tả hệ thống nhúng đơn giản mà bạn dùng hằng ngày (theo mẫu Mục 4).

Tiêu chí đánh giá buổi thực hành

Tiêu chí	Điểm
Cài đặt môi trường thành công	2
Bài Lab 1 chạy đúng yêu cầu	5
Báo cáo rõ ràng, đầy đủ	2
Hoàn thành đúng thời gian	1

Hết Tuần 1

Hẹn gặp lại ở Tuần 2: Kiến trúc & tập lệnh vi điều khiển AVR